Python Basics Review

What is Python?

- **Introduction**: Python is a general-purpose programming language known for its simplicity and ease of use. Python is used in many fields like data science and machine learning, web development, scripting and automation, embedded systems and IoT, and much more.
- **Common Use Cases**: Python is used in data science, machine learning, web development, cybersecurity, automation and microcomputers like the Raspberry Pi and MicroPython-compatible boards.

Variables

- **Declaring Variables**: To declare a variable, you start with the variable name followed by the assignment operator (`=`) and then the value. This can be a number, string, boolean, etc. Here are some examples:

```
name = 'John Doe'
age = 25
```

- **Naming Conventions for Variables**: Here are the naming conventions you should use for variables:
  - Variable names can only start with a letter or an underscore (_), not a number.
  - Variable names can only contain alphanumeric characters (a-z, A-Z, 0-9) and underscores (_).
  - Variable names are case-sensitive — `age`, `Age`, and `AGE` are all considered unique.
  - Variable names cannot be one of Python's reserved keywords such as `if`, `class`, or `def`.
  - Variables names with multiple words are separated by underscores. Ex. `snake_case`.

Comments

- **Single Line Comments**: These types of comments should be used for short notes you wish to leave in your code.

```
# This is a single line comment
```

- **Multi-line Comments**: You can use a `#` at the beginning of each line to create comments across multiple lines.

```
# This is a multi-line comment.
# Here is some code commented out.
#
# name = 'John Doe'
# age = 25
```

- `print()` **Function**: To print data to the terminal, you can use the `print()` function like this:

```
print('Hello world!') # Hello world!
```

Common Data Types in Python

- **Introduction**: Python is a dynamically typed language, meaning you don't need to declare a variable's data type. Python determines the type from the value assigned to the variable.
- **Integer**: A whole number without decimals:

Success! You have signed in to your account. Happy Coding!

```
my_float_var = 4.50
print('Float:', my_float_var) # Float: 4.5
```

- **String**: A sequence of characters wrapped in quotes:

```
my_string_var = 'hello'
print('String:', my_string_var) # String: hello
```

- **Boolean**: A value representing either `True` or `False`:

```
my_boolean_var = True
print('Boolean:', my_boolean_var) # Boolean: True
```

- **Set**: An unordered collection of unique elements:

```
my_set_var = {7, 5, 8}
print('Set:', my_set_var) # Set: {7, 5, 8}
```

- **Dictionary**: A collection of key-value pairs, enclosed in curly braces:

```
my_dictionary_var = {"name": "Alice", "age": 25}
print('Dictionary:', my_dictionary_var) # Dictionary: {'name': 'Alice', 'age': 25}
```

- **Tuple**: An immutable ordered collection, enclosed in parentheses:

```
my_tuple_var = (7, 5, 8)
print('Tuple:', my_tuple_var) # Tuple: (7, 5, 8)
```

- **Range**: A sequence of numbers, often used in loops:

```
my_range_var = range(5)
print(my_range_var) # range(0, 5)
```

- **List**: An ordered collection of elements that supports different data types:

```
my_list = [22, 'Hello world', 3.14, True]
print(my_list) # [22, 'Hello world', 3.14, True]
```

- **None**: A special value that represents the absence of a value:

```
my_none_var = None
print('None:', my_none_var) # None: None
```

Immutable and Mutable Types

- **Immutable Types**: These types cannot change once declared, although you can point their variables at something new, which is called reassignment. They include integer, float, boolean, string, tuple, range, and `None`.
- **Mutable Types**: These types can change once declared. You can add, remove, or update their items. They include collection types such as list, set, and dictionary.

Success! You have signed in to your account. Happy Coding!

```
print(type(greeting)) # <class 'str'>
print(type(age)) # <class 'int'>
```

- `isinstance()` **Function**: This is used to check if a variable matches a specific data type:

```
greeting = 'Hello world'
name = 'John Doe'

print(isinstance(greeting, str)) # True
print(isinstance(name, int)) # False
```

Working with Strings

- **Definition**: Strings are immutable, which means you cannot change them after they have been created. In Python, you can use either single or double quotes. Choose one style and use it consistently:

```
developer = 'Jessica'
city = "Los Angeles"
```

- **Accessing Characters from Strings**: You can access characters from strings by using bracket notation like this:

```
my_str = 'Hello world'

print(my_str[0])  # H
print(my_str[6])  # w

print(my_str[-1])  # d
print(my_str[-2]) # l
```

- **Escaping Strings**: You can use a backslash (`\`) if your string contains quotes like this:

```
msg = 'It\'s a sunny day'
quote = "She said, \"Hello!\""
```

- **String Concatenation**: To concatenate strings, you can use the `+` operator like this:

```
developer = 'Jessica'
print('My name is ' + developer + '.') # My name is Jessica.
```

Another way to concatenate strings is by using the `+=` operator. This is used to perform concatenation and assignment in the same step like this:

```
greeting = 'My name is '
developer = 'Jessica.'

greeting += developer
print(greeting) # My name is Jessica.
```

Success! You have signed in to your account. Happy Coding!

```
print(greeting) # My name is Jessica.
```

- **String Slicing**: This is when you can extract portions of a string. Here is the basic syntax:

```
str[start:stop:step]
```

The start position represents the index where the extraction should begin. The stop position is where the slice should end. This position is non inclusive. The step position represents the interval to increment for the slicing. Here are some examples:

```
message = 'Python is fun!'

print(message[0:6])  # Python
print(message[7:])  # is fun!
print(message[::2])  # Pto sfn
```

- **Getting the Length of a String**: The `len()` function is used to return the number of the characters in the string:

```
developer = 'Jessica'

print(len(developer)) # 7
```

Working with the `in` operator

- `in` **Operator**: This returns a boolean that specifies whether the character or characters exist in the string or not:

```
my_str = 'Hello world'

print('Hello' in my_str)  # True
print('hey' in my_str)    # False
print('hi' in my_str)    # False
print('e' in my_str)  # True
print('f' in my_str)  # False
```

Common String Methods

- `str.upper()`: This returns a new string with all characters converted to uppercase:

```
developer = 'Jessica'

print(developer.upper()) # JESSICA
```

- `str.lower()`: This returns a new string with all characters converted to lowercase:

```
developer = 'Jessica'

print(developer.lower()) # jessica
```

- `str.strip()`: This returns a copy of the string with specified leading and trailing characters removed (if no argument is passed to the method, it removes leading and trailing whitespace).

Success! You have signed in to your account. Happy Coding!

- `replace()`: This returns a new string with all occurrences of the old string replaced by a new one.

```
greeting = 'hello world'

replaced_my_str = greeting.replace('hello', 'hi')
print(replaced_my_str)  # 'hi world'
```

- `split()`: This is used to split a string into a list using a specified separator. A separator is a string specifying where the split should happen.

```
dashed_name = 'example-dashed-name'

split_words = dashed_name.split('-')
print(split_words)  # ['example', 'dashed', 'name']
```

- `join()`: This is used to join elements of an iterable into a string with a separator. An iterable is a collection of elements that can be looped over like a list, string or a tuple.

```
example_list = ['example', 'dashed', 'name']

joined_str = ' '.join(example_list)
print(joined_str)  # example dashed name
```

- `str.startswith(prefix)`: This returns a boolean indicating if a string starts with the specified prefix:

```
developer = 'Naomi'

result = developer.startswith('N')
print(result)  # True
```

- `str.endswith(suffix)`: This returns a boolean indicating if a string ends with the specified suffix:

```
developer = 'Naomi'

result = developer.endswith('N')
print(result)  # False
```

- `str.find()`: This returns the index for the first occurrence of a substring. If one is not found, then `-1` is returned:

```
developer = 'Naomi'

result = developer.find('N')
print(result)  # 0

city = 'Los Angeles'
print(city.find('New')) # -1
```

- `str.count(substring)`: This counts how many times a substring appears in a string:

Success! You have signed in to your account. Happy Coding!

```
dessert = 'chocolate cake'
print(dessert.capitalize()) # Chocolate cake
```

- `str.isupper()`: This returns `True` if all letters in the string are uppercase and `False` if otherwise:

```
dessert = 'chocolate cake'
print(dessert.isupper()) # False
```

- `str.islower()`: This returns `True` if all letters in the string are lowercase and `False` if otherwise:

```
dessert = 'chocolate cake'
print(dessert.islower()) # True
```

- `str.title()`: This returns a new string with the first letter of each word capitalized:

```
city = 'los angeles'
print(city.title()) # Los Angeles
```

- `str.maketrans()`: This method is used to create a table of 1 to 1 character mappings for translation. It is often used with the `translate()` method which applies that table to a string and return the translated result.

```
trans_table = str.maketrans('abc', '123')
print(trans_table) # {97: 49, 98: 50, 99: 51}

result = 'abcabc'.translate(trans_table)
print(result)  # 123123
```

Common Operations used with Integers and Floats

- **Basic Math Operations**: In Python, you can do basic math operations with integers and floats including addition, subtraction, multiplication and division:

```
int_1 = 56
int_2 = 12
float_1 = 5.4
float_2 = 12.0

# Addition

print('Integer Addition:', int_1 + int_2) # Integer Addition: 68
print('Float Addition:', float_1 + float_2) # Float Addition: 17.4

# Subtraction

print('Int Subtraction:', int_1 - int_2) # Int Subtraction: 44
print('Float Subtraction:',  float_2 - float_1) # Float Subtraction: 6.6

# Multiplication
```

Success! You have signed in to your account. Happy Coding!

```
print('Division:', int_1 / int_2) # Division: 4.666666666666667
print('Float Division:', float_2 / float_1) # Float Division: 2.222222222222222
```

When you add a float and an integer, the result will be converted to a float like this:

```
int_1 = 56
float_1 = 5.4

print(int_1 + float_1) # 61.4
```

- **Modulo Operator (`%`)**: This returns the remainder when a number is divided by another number:

```
int_1 = 56
int_2 = 12

print(int_1 % int_2) # 8
```

- **Floor Division (`//`)**: This operator is used to divide two numbers and round down the result to the nearest whole number:

```
int_1 = 56
int_2 = 12

print(int_1 // int_2) # 4
```

- **Exponentiation Operator (`**`)**: This operator is used to raise a number to the power of another:

```
int_1 = 4
int_2 = 2

print(int_1 ** int_2) # 16
```

- **`float()` Function**: You can use this function to convert an integer to float.

```
num = 4

print(float(num)) # 4.0
```

- **`int()` Function**: You can use this function to convert an float to an integer.

```
num = 4.0

print(int(num)) # 4
```

- **`round()` Function**: This is used to round a number to the nearest whole integer:

```
num_1 = 3.4
num_2 = 7.7
```

Success! You have signed in to your account. Happy Coding!

```
num = -13

print(abs(num)) # 13
```

- `pow( )` **Function**: This is used to raise a number to the power of another:

```
result = pow(2, 3)
print(result)  # 8
```

Augmented Assignments

- **Definition**: Augmented assignment applies an operation to a variable and stores the result back in the same variable, all in one step.

```
# Addition assignment
my_var = 10
my_var += 5

print(my_var) # 15

# Subtraction assignment
count = 14
count -= 3

print(count) # 11

# Multiplication assignment
product = 65
product *= 7

print(product) # 455

# Division assignment
price = 100
price /= 4

print(price) # 25.0

# Floor Division assignment
total_pages = 23
total_pages //= 5

print(total_pages) # 4

# Modulo assignment
bits = 35
bits %= 2

print(bits) # 1
```

Success! You have signed in to your account. Happy Coding!

Working with Functions

- **Definition**: Functions are reusable pieces of code that take inputs (arguments) and returns an output. To call a function, you need to reference the function name followed by a set of parenthesis:

```
# Defining a function

def get_sum(num_1, num_2):
    return num_1 + num_2

result = get_sum(3, 4) # function call
print(result) # 7
```

If a function does not explicitly return a value, then the default return value is `None`:

```
def greet():
    print('hello')

result = greet() # hello
print(result) # None
```

You can also supply default values to parameters like this:

```
def get_sum(num_1, num_2=2):
    return num_1 + num_2

result = get_sum(3)
print(result) # 5
```

If you call the function without the correct number of arguments, you will get a `TypeError`:

```
def calculate_sum(a, b):
    print(a + b)

calculate_sum()

# TypeError: calculate_sum() missing 2 required positional arguments: 'a' and 'b'
```

Common Built-in Functions

- **`input()` Function**: This is used to prompt the user for some input:

```
name = input('What is your name?') # User types 'Kolade' and presses Enter
print('Hello', name) # Hello Kolade
```

- **`int()` Function**: This is used to convert a number, boolean, or a numeric string into an integer:

Success! You have signed in to your account. Happy Coding!

Scope in Python

- **Local Scope**: This is when a variable declared inside a function can only be accessed within that function.

```
def my_func():
    num = 10
    print(num)
```

- **Enclosing Scope**: This is when a function that's nested inside another function can access the variables of the function it's nested within.

```
def outer_func():
    msg = 'Hello there!'

    def inner_func():
        print(msg)
    inner_func()

print(outer_func()) # Hello there!
```

- **Global Scope**: This refers to variables that are declared outside any function and can be accessed from anywhere in the program.

```
tax = 0.70

def get_total(subtotal):
    total = subtotal + (subtotal * tax)
    return total

print(get_total(100))  # 170.0
```

- **Built-in Scope**: Names that Python provides, including built-in functions, modules, and keywords. These names are available anywhere in your program.

```
print(str(45)) # '45'
print(type(3.14)) # <class 'float'>
print(isinstance(3, str)) # False
```

Comparison Operators

- **Equal (`==`)**: Checks if two values are equal:

```
print(3 == 4) # False
```

- **Not equal (`!=`)**: Checks if two values are not equal:

```
print(3 != 4) # True
```

- **Strictly greater than (`>`)**: Checks if one value is greater than another:

Success! You have signed in to your account. Happy Coding!

```
print(3 < 4) # True
```

- **Greater than or equal(`>=`)**: Checks if one value is greater than or equal to another:

```
print(3 >= 4) # False
```

- **Less than or equal(`<=`)**: Checks if one value is less than or equal to another:

```
print(3 <= 4) # True
```

Working with `if`, `elif` and `else` Statements

- **`if` Statements**: These are conditions used to determine if something is true or not. If the condition evaluates to `True`, then that block of code will run.

```
age = 18

if age >= 18:
    print('You are an adult') # You are an adult
```

- **`elif` Clause**: These are conditions that come after an `if` statement. An `elif` clause runs only if all previous conditions evaluate to `False` and its own condition evaluates to `True`.

```
age = 16

if age >= 18:
    print('You are an adult')
elif age >= 13:
    print('You are a teenager')  # You are a teenager
```

- **`else` Clause**: This will run if no other conditions evaluate to `True`.

```
age = 12

if age >= 18:
    print('You are an adult')
elif age >= 13:
    print('You are a teenager')
else:
    print('You are a child')  # You are a child
```

You can also use nested `if` statements like this:

```
is_citizen = True
age = 25

if is_citizen:
    if age >= 18:
        print('You are eligible to vote') # You are eligible to vote
```

Success! You have signed in to your account. Happy Coding!

- **Definition**: In Python, every value has an inherent boolean value, or a built-in sense of whether it should be treated as `True` or `False` in a logical context. Many values are considered truthy, that is, they evaluate to `True` in a logical context. Others are falsy, meaning they evaluate to `False`. Here are some examples of falsy values:

```
None
False
Integer 0
Float 0.0
Empty strings ''
```

Other values like non-zero numbers, and non-empty strings are truthy.

Working with the `bool()` Function

- **Definition**: If you want to check whether a value is truthy or falsy, you can use the built-in `bool()` function. It explicitly converts a value to its boolean equivalent and returns `True` for truthy values and `False` for falsy values. Here are a few examples:

```
print(bool(False)) # False
print(bool(0))  # False
print(bool('')) # False

print(bool(True)) # True
print(bool(1)) # True
print(bool('Hello')) # True
```

Boolean Operators and Short-circuiting

- **Definition**: These are special operators that allow you to combine multiple expressions to create more complex decision-making logic in your code. There are three Boolean operators in Python: `and`, `or`, and `not`.
- `and` **Operator**: This operator takes two operands and returns the first operand if it is falsy, otherwise, it returns the second operand. Both operands must be truthy for an expression to result in a truthy value.

```
is_citizen = True
age = 25

print(is_citizen and age) # 25
```

You can also use the `and` operator in conditionals like this:

```
is_citizen = True
age = 25

if is_citizen and age >= 18:
    print('You are eligible to vote') # You are eligible to vote
else:
    print('You are not eligible to vote')
```

- `or` **Operator**: This operator returns the first operand if it is truthy, otherwise, it returns the second operand. An `or` expression results in a truthy value if at least one operand is truthy. Here is an example:

Success! You have signed in to your account. Happy Coding!

Just like with the `and` operator, you can use the `or` operator in conditionals like this:

```
age = 19
is_student = True

if age < 18 or is_student:
    print('You are eligible for a student discount') # You are eligible for a student discount
else:
    print('You are not eligible for a student discount')
```

- **Short-circuiting**: The `and` and `or` operators are known as a short-circuit operators. Short-circuiting means Python checks values from left to right and stops as soon as it determines the final result.
- `not` **Operator**: This operator takes a single operand and inverts its boolean value. It converts truthy values to `False` and falsy values to `True`. Unlike the previous operators we looked at, `not` always returns `True` or `False`. Here are some examples:

```
print(not '') # True, because empty string is falsy
print(not 'Hello') # False, because non-empty string is truthy
print(not 0) # True, because 0 is falsy
print(not 1) # False, because 1 is truthy
print(not False) # True, because False is falsy
print(not True) # False, because True is truthy
```

Here is an example of the `not` operator in a conditional:

```
is_admin = False

if not is_admin:
    print('Access denied for non-administrators.') # Access denied for non-administrators.
else:
    print('Welcome, Administrator!')
```

Assignment

Review the Python Basics topics and concepts.

Please complete the assignment

Submit

Ask for Help